Asynchronous HTTP & WebSockets

Redes e Serviços

Daniel Corujo, Paulo Salvador, Miguel Matos

March 2026

Introduction

In the previous lab, you used `curl` to send HTTP requests and watched each exchange complete before the next one began — your client sent a request, blocked until the server responded, and moved on. This model is simple and predictable, but it has limits. What happens when a server must handle hundreds of simultaneous connections? What if a client needs the server to send data without being asked?

This lab explores two answers to those questions. What you will read here extends the concepts from the previous lab to cover concurrency and persistent connections.

A. Synchronous vs Asynchronous I/O

When a FastAPI endpoint is declared as a regular `def` function, it runs in a thread pool. Each call occupies a thread until it returns — if the operation involves waiting (a slow database query, a remote API call, or an explicit sleep), that thread is **idle and unavailable** for other requests.

When an endpoint is declared `async def`, it runs on the **event loop** — a single-threaded scheduler that switches between tasks whenever one of them awaits something. An `await asyncio.sleep(5)` does not block the thread; it signals the event loop to come back in 5 seconds and run other tasks in the meantime. For I/O-bound work, this allows one process to serve many concurrent requests without additional threads.

From the client's perspective, both `def` and `async def` produce identical HTTP responses. The difference is entirely within the server.

B. From HTTP to WebSockets

Every HTTP exchange you have made so far follows a strict pattern: the **client** initiates a request, the server responds, and the exchange is complete. The server can never send data unless the client asks first. This is the **request/response model**.

WebSockets break this constraint. A WebSocket connection begins as an ordinary HTTP `GET` request carrying two extra headers:

```
Upgrade: websocket
Connection: Upgrade
```

If the server supports WebSockets, it replies with `101 Switching Protocols` and the connection is transformed. The TCP connection remains open, but it is no longer HTTP — **either side may send a message at any time**. This is what makes real-time applications possible: live dashboards, notifications, and collaborative tools where the server needs to push updates without waiting to be asked.

C. The Event Loop and Long-Lived Connections

Because WebSocket connections are long-lived, they coexist with regular HTTP connections inside the same FastAPI process. A single `uvicorn` worker uses an event loop to manage all of them simultaneously: incoming HTTP requests, open WebSocket connections, and background coroutines — all interleaved on one thread.

You will observe this directly with `netstat`: each open WebSocket connection appears as one `ESTABLISHED` TCP entry, held open between data pushes. This is unlike the short-lived HTTP connections from the previous lab, where each `curl` command created and closed a connection within seconds.

Exercises

Practical Lab: Async HTTP & WebSockets

Objective: Observe the difference between synchronous and asynchronous HTTP handling, understand the WebSocket protocol upgrade at the packet level, and interact with a locally deployed real-time monitoring API using both HTTP and WebSocket clients.

Use ‘`docker compose` to deploy the two services we will be using during the class inside the class VM:

```
$ docker compose up -d --build
```

Port `8001` is the **demo** service used in Parts 1 and 2. Port `8000` is a sample **monitoring API** to be used in Part 3.

Part 1: Async HTTP

1.1 Blocking vs Non-Blocking

The demo service exposes two endpoints that each take 5 seconds to respond:

- `GET /slow-sync` — implemented with `time.sleep(5)` inside a regular `def` function
- `GET /slow-async` — implemented with `await asyncio.sleep(5)` inside an `async def` function

Fire both requests at the synchronous endpoint simultaneously using & (background execution in bash):

```
$ curl http://localhost:8001/slow-sync & curl http://localhost:8001/slow-sync
```

Note the total elapsed time. Now repeat with the asynchronous endpoint:

```
$ curl http://localhost:8001/slow-async & curl http://localhost:8001/slow-async
```

> **Thinking Question:** Both endpoints produce identical HTTP responses. From the client’s perspective, is there any way to tell which one the server used? If not, what does this tell you about where the benefit of `async` is realised?

1.2 Background Tasks

FastAPI’s `BackgroundTasks` mechanism allows an endpoint to return a response immediately while continuing work afterwards. The demo service includes a `POST /submit` endpoint that queues a task and returns `202 Accepted` before the work is done, and a `GET /status` endpoint that reports how many tasks have completed.

```
# Submit a task — should return 202 immediately
$ curl -v -X POST http://localhost:8001/submit

# Check the status before the background task finishes
$ curl http://localhost:8001/status
```

Wait approximately 4 seconds, then check the status again:

```
$ curl http://localhost:8001/status
```

> **Thinking Question:** HTTP status `202 Accepted` means “I received your request but have not finished processing it.” How is this different from `200 OK`?

1.3 Async is Invisible on the Wire

Find the Docker bridge interface for the running containers:

```
$ docker network ls
```

Start a Wireshark capture on that interface (e.g., `br-a1b2c3d4e5f6`) with display filter `http`.

Send a single request to the asynchronous endpoint:

```
$ curl http://localhost:8001/slow-async
```

Find the HTTP `GET /slow-async` request packet and the `200 OK` response packet. Note the timestamps.

> **Thinking Question:** Wireshark shows approximately 5 seconds of silence between the request and the response. This looks identical regardless of whether the server uses `async def` or `def`. What does this confirm about asynchronous I/O — is it a network-level concept or a server-internal one?

Part 2: WebSockets

2.1 Installing websocat

`websocat` is a command-line WebSocket client — the equivalent of `curl` for WebSocket connections.

It does not have an official Debian package, but you can still download its binary and install it manually:

```
$ wget https://github.com/vi/websocat/releases/download/v1.14.1/websocat.x86_64-unknown-linux-musl
$ chmod +x websocat.x86_64-unknown-linux-musl
$ sudo mv websocat.x86_64-unknown-linux-musl /usr/local/bin/websocat
```

Basic usage:

```
# Connect to a WebSocket endpoint
$ websocat ws://localhost:8001/ws/echo

# Type a message and press Enter to send
# Press Ctrl+C to disconnect
```

2.2 The HTTP Upgrade Handshake

A WebSocket connection starts as a normal HTTP GET request with two additional headers. When the server accepts the upgrade, it replies with `101 Switching Protocols` and the connection is transformed.

Client sends:

```
GET /ws/echo HTTP/1.1
Host: localhost:8001
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

Server responds:

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

After the `101`, neither side speaks HTTP anymore. The `Sec-WebSocket-Accept` value is derived from the client's key using SHA-1 — a challenge-response mechanism to confirm the server intentionally accepted the upgrade.

2.3 First WebSocket Connection

The demo service includes a `/ws/echo` endpoint that echoes back any text you send.

Start a Wireshark capture on the Docker bridge interface with display filter `http or websocket`. Then connect with websocat:

```
$ websocat ws://localhost:8001/ws/echo
```

Type a few messages and observe the echo responses. Then disconnect with Ctrl+C.

In Wireshark, examine the capture:

1. Find the `HTTP 101 Switching Protocols` packet. Expand the HTTP layer and locate the `Upgrade` and `Sec-WebSocket-Accept` headers.
2. Find the WebSocket data frames that follow. For each frame, identify the `Opcode` field (`Text`), the `Payload length`, and read the decoded payload — you should see your messages and their echoes.
3. Find the connection close event. Is it a graceful WebSocket `Close` frame (opcode `0x8`) or an abrupt TCP reset?

> **Thinking Question:** The payload content of WebSocket frames is visible in plain text in Wireshark. What does this imply for a production WebSocket deployment?

2.4 Multiple Connections

Open **two** websocat sessions to the echo endpoint simultaneously:

```
# Terminal 1
$ websocat ws://localhost:8001/ws/echo

# Terminal 2
$ websocat ws://localhost:8001/ws/echo
```

Type a message in Terminal 1. Only Terminal 1 sees the echo — Terminal 2 receives nothing.

While both sessions are open, in a third terminal check the connection table:

```
$ netstat -tn | grep 8001
```

> **Thinking Question:** The echo server handles each connection independently — there is no shared state between clients. If you wanted to build an application where every connected client receives the same server-generated data, what would the server need to maintain?

Part 3: Deploying the Environmental Monitoring API

The monitoring API is already running on port `8000` — it was started alongside the demo service in Part 1.1. It simulates air quality sensor readings for four stations — **Aveiro, Lisboa, Porto, Coimbra** — and pushes live updates to connected WebSocket clients every 3 seconds.

The API exposes the following endpoints:

Method	Endpoint	Description
GET	`/stations`	List all stations with their latest reading
GET	`/stations/{station}`	Get a station's reading history (last 20 entries)
POST	`/stations/{station}/alert`	Queue an alert (requires authorisation)
WebSocket	`/ws/{station}`	Subscribe to live readings for one station
WebSocket	`/ws`	Subscribe to live readings for all stations

Open `http://localhost:8000/docs` in your VM's browser. FastAPI generates an interactive Swagger UI for the HTTP endpoints. Notice that the WebSocket endpoints do not appear there — Swagger UI has no way to test persistent connections.

3.1 Exploring the HTTP Endpoints

```
# List all stations and their most recent readings
$ curl http://localhost:8000/stations | jq .

# Get the full reading history for Aveiro
$ curl http://localhost:8000/stations/aveiro | jq .

# Try a station that does not exist
$ curl -v http://localhost:8000/stations/braga

# Queue an alert without authorisation — should return 401
$ curl -v -X POST http://localhost:8000/stations/porto/alert

# Queue an alert with the correct token
$ curl -v -X POST http://localhost:8000/stations/porto/alert \
    -H "Authorization: Bearer secret-token"
```

> **Thinking Question:** The `/stations` endpoint is declared `async def` but performs no I/O — it simply returns an in-memory list. Would there be any observable difference if it were `def` instead? When does `async def` actually matter for a FastAPI endpoint?

3.2 Subscribing to Live Readings

Connect to a station's WebSocket endpoint. You do not need to send anything — the server pushes readings automatically every 3 seconds:

```
# Terminal 1 — subscribe to Aveiro
$ websocat ws://localhost:8000/ws/aveiro
```

Open a second terminal and subscribe to all stations simultaneously:

```
# Terminal 2 — subscribe to all stations
$ websocat ws://localhost:8000/ws
```

Leave both sessions running for at least 15 seconds. Observe that Terminal 1 receives only Aveiro readings, while Terminal 2 receives readings from all four stations interleaved.

After disconnecting both sessions, retrieve the history via HTTP:

```
$ curl http://localhost:8000/stations/aveiro | jq .
```

The history should contain the readings that were pushed over the WebSocket while you were connected.

> **Thinking Question:** The server sends data every 3 seconds without any client request. How would you achieve the same behaviour using plain HTTP, without WebSockets? What are the trade-offs of that approach?

3.3 Observing Connections with netstat

While both websocat sessions from Part 3.2 are open, in a separate terminal:

```
$ netstat -tn | grep 8000
```

Note the number of `ESTABLISHED` entries. Disconnect one session with Ctrl+C, then run `netstat` again.

> **Thinking Question:** Each open WebSocket connection appears as one `ESTABLISHED` TCP entry held open between data pushes. In the previous lab, each `curl` request also briefly appeared as `ESTABLISHED` but disappeared within seconds. What is the fundamental difference in the lifecycle of these two types of connections?

3.4 Capturing WebSocket Traffic in Wireshark

Start a Wireshark capture on the Docker bridge interface with display filter `tcp.port == 8000`. Generate the following traffic in sequence:

```
# 1. An HTTP request
$ curl http://localhost:8000/stations

# 2. Connect one WebSocket client and leave it running for ~15 seconds
$ websocat ws://localhost:8000/ws/aveiro

# 3. Connect a second client in another terminal
$ websocat ws://localhost:8000/ws/porto

# 4. Disconnect one client with Ctrl+C

# 5. An HTTP history request after the WebSocket session
$ curl http://localhost:8000/stations/aveiro | jq .
```

Stop the capture and answer the following:

1. How many `HTTP 101 Switching Protocols` responses appear? Confirm there is one per WebSocket client.
2. Find the server-initiated WebSocket frames arriving every 3 seconds. Note that no client message precedes them. What does this confirm about the direction of data flow?
3. When a client disconnected with Ctrl+C, is the termination a graceful WebSocket `Close` frame or a TCP reset? What would you expect the server to do when it detects this condition?
4. Find both the WebSocket frames for Aveiro and the subsequent `HTTP 200 OK` response for `GET /stations/aveiro`. Do they carry the same data?

Cleanup

```
$ docker compose down
```